

6교시: Docker Compose로 전체 서비스 실행

수업 목표

- compose up, ps, logs, 브라우저 접속, 서비스 상태 확인을 MSA 운영 문제로 설명한다.
- 서비스 경계, 네트워크, 설정, 로그, health 중 최소 두 가지 evidence로 정상 상태를 판단한다.
- 장애가 나면 재현, 관찰, 가설, 수정, 재확인 순서로 기록한다.

선행 지식

이미 알고 있어야 할 것	오늘 다시 연결할 것
Docker image/container	MSA의 서비스별 실행 단위
Docker Compose service	서비스 토폴로지와 의존성
port binding, service name DNS	내부 통신과 외부 진입점 구분
logs/status evidence	분산 로그와 장애 분석

50분 흐름

시간	활동	학습 초점	학생 산출
0-5분	문제 상황 제시	왜 단일 앱보다 운영이 어려워지는지 본다.	오늘의 질문 1개
5-15분	핵심 개념 설명	Docker Compose로 전체 서비스 실행의 운영 의미를 정리한다.	용어 note
15-25분	구조/공식 기준 확인	service, dependency, health, logs를 공식 용어와 연결한다.	판단 표
25-40분	실습 또는 데모	Compose 상태와 로그로 실제 증거를 확인한다.	command evidence
40-47분	장애/비용/보안 점검	운영 위험과 복구 기준을 정리한다.	risk note
47-50분	다음 교시 연결	다음 service boundary로 확장한다.	handoff 문장

시작 상황

서비스가 하나일 때는 container가 켜졌는가 , port가 열렸는가 , HTTP 응답이 오는가 를 보면 대부분의 상태를 파악할 수 있다. MSA에서는 같은 질문을 서비스마다 반복해야 하고, 한 서비스의 정상 상태가 전체 사용자 경험의 정상 상태를 보장하지 않는다.

오늘의 주제인 Docker Compose로 전체 서비스 실행 은 이 복잡도를 줄이기 위한 관찰 기준을 만든다. 인프라/DevOps 엔지니어는 내부 구현을 모두 알 필요는 없지만, 어떤 서비스가 어떤 주소로 누구에게 요청하고, 어떤 설정을 필요로 하며, 어디에서 실패 증거를 남기는지는 알아야 한다.

Generated Visual: 6교시 전용 인포그래픽

Week 3 Day 1 Lesson 6

Docker Compose로 전체 서비스 실행

단 한 번의 명령으로 개발 스택 전체를 빌드하고 실행합니다.

명령 실행 흐름

1 전체 서비스 빌드

build

\$ docker compose up --build -d

이미지가 없으면 빌드하고, 컨테이너를 데몬 모드로 실행

2 실행 중인 서비스 확인

ps

\$ docker compose ps

모든 서비스의 상태와 포트를 확인

3 API 상태 확인 (헬스 체크)

curl

\$ curl http://localhost:8080/api/status

프론트엔드를 통해 API /api/status 엔드포인트 호출

4 로그 확인

logs

\$ docker compose logs -f --tail=100

모든 서비스 로그를 실시간으로 확인

5 종료 및 정리 (필요 시)

cleanup

\$ docker compose down

\$ docker compose down -v

컨테이너 중지 및 네트워크/볼륨 정리 (-v 는 볼륨까지 삭제)

서비스 상태 (docker compose ps 결과)

frontend

running

포트: 3000:80

역할: Web UI

api

healthy

포트: 8080:8080

역할: API Server

db

healthy

포트: 5432:5432

역할: PostgreSQL

worker

running

(내부 포트)

역할: 백그라운드 작업

docker compose ps (예시)

NAME	IMAGE	COMMAND	SERVICE	STATUS	PORTS
frontend	paperclip-frontend	"nginx -g 'daemon off;'"	frontend	running	0.0.0.0:3000->80/tcp
api	paperclip-api	"uvicorn main:app"	api	runlthy	0.0.0.0:8080->8080/tcp
db	postgres:15	"docker-entrypoint.s..."	db	healthy	0.0.0.0:5432->5432/tcp
worker	paperclip-worker	"python worker.py"	worker	running	

curl /api/status (예시)

```
{
  "status": "ok",
  "service": "api",
  "version": "1.0.0",
  "timestamp": "2025-05-20T09:30:02"
}
```

docker compose logs -f (예시)

```
api      | INFO:   Started server process [1]
api      | INFO:   Application startup complete.
db       | 2025-05-20 09:30:00 LOG:  database system is ready
worker  | [2025-05-20 09:30:05] Job queue connected
frontend | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty
```

빠른 명령 요약

1. build & up

2. ps (상태 확인)

3. curl (상태 체크)

4. logs (로그 확인)

5. cleanup (종료/정리)

TIP

- d 옵션: 데몬(백그라운드) 모드로 실행
- build 옵션: 로컬 변경사항을 반영하여 이미지 재빌드
- ps: 현재 상태와 포트 매핑을 빠르게 확인
- logs -f: 실시간 로그 스트리밍으로 문제 빠르게 파악
- down -v: 볼륨까지 정리하여 깨끗한 상태로 재시작

이 이미지는 Week 3 Day 1 6교시 Docker Compose로 전체 서비스 실행 의 핵심 개념을 세션 전용으로 시각화한 자료다. 다른 교시와 같은 그림을 재사용하지 않고, 이 세션에서 확인할 운영 evidence와 판단 기준에 맞춰 읽는다.

Visual 1: 서비스 토폴로지 읽기

```
graph LR
    User[User / Browser] --> frontend[frontend<br/>external port 18083]
    frontend --> api[api<br/>internal port 8080]
    api --> database[database<br/>PostgreSQL 5432]
    worker[worker] --> api
    worker --> database
```

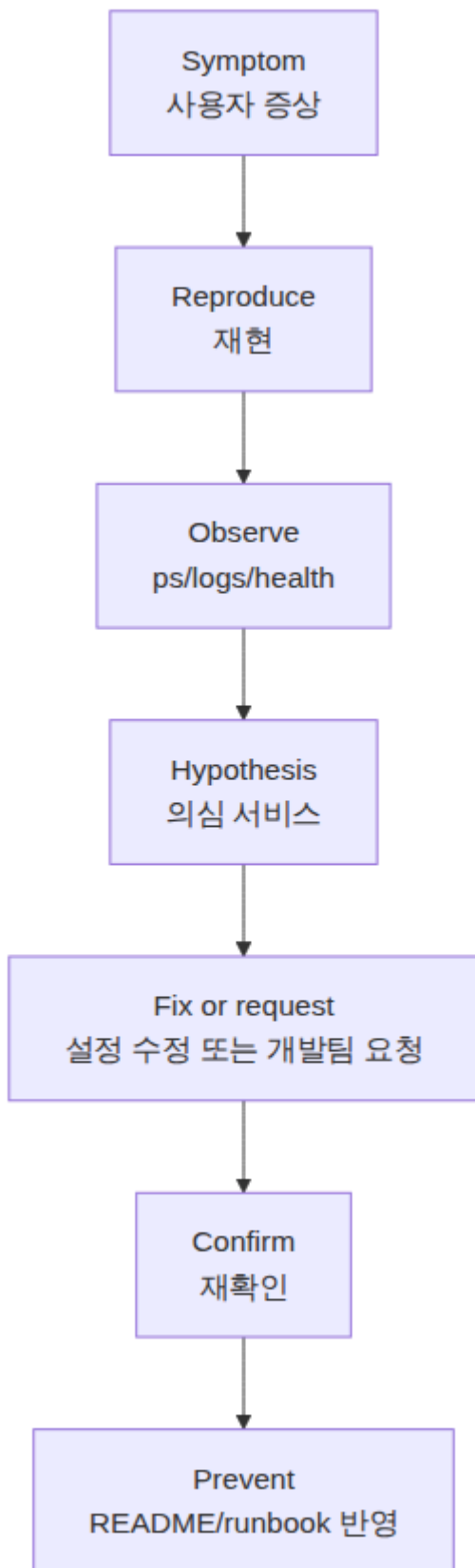
읽는 순서: 사용자는 frontend로 들어오고, frontend는 api로 요청을 넘긴다. api와 worker는 database에 의존한다. 이 그림에서 중요한 것은 박스 개수가 아니라, 장애가 어디에서 어디로 전파될 수 있는지 보는 것이다.

Visual 2: 운영 판단 표

확인 대상	정상 evidence	실패 증상	다음 행동
frontend	browser 접속, HTTP 200	화면은 뜨지만 API 데이터 없음	proxy/API URL 확인
api	/health, /api/status 응답	5xx, timeout, DB unavailable	env, DB host, logs 확인
worker	주기적 처리 로그	처리 정지, 반복 실패	dependency와 retry 주기 확인
database	healthcheck healthy	connection refused	volume, credential, readiness 확인

이 표는 기능 목록이 아니라 장애 분석 순서다. 운영자는 사용자가 본 증상에서 시작해 어느 서비스 evidence를 먼저 볼지 정해야 한다.

Visual 3: 장애 분석 흐름



이 흐름은 Week 1의 RCA와 같다. 달라진 점은 관찰 대상이 하나의 process가 아니라 여러 service의 조합이라는 점이다.

실습 기준

repository root에서 시작한다. Day 1 이후 모든 교시는 같은 실습 앱을 기준으로 진행한다.

```
cd week3/day1/labs/msa-demo
docker compose ps
docker compose logs --tail=40 api
docker compose logs --tail=40 worker
curl -s http://localhost:18083/api/status
```

기대 결과는 서비스 상태가 모두 `running` 또는 `healthy` 에 가깝고, `/api/status` 가 frontend를 통해 api 상태를 JSON으로 돌려주는 것이다. 실패하면 실패 자체를 evidence로 남긴다.

DevOps 원칙 연결

관점	오늘의 연결
비용 절감	로컬 Compose로 topology를 먼저 검증해 cloud 리소스 생성 전 오류를 줄인다.
개발/배포 효율	서비스별 실행 계약을 문서화해 개발팀과 운영팀의 왕복 질문을 줄인다.
관리 효율	health, logs, dependency map을 표준화해 장애 triage 시간을 줄인다.

흔한 오해

오해	바로잡기
MSA는 서비스를 많이 나누면 자동으로 좋아진다.	서비스 경계가 운영 비용보다 큰 이익을 줄 때 선택한다.
frontend가 보이면 전체 시스템이 정상이다.	API나 DB 장애가 숨어 있을 수 있으므로 service별 health를 봐야 한다.
depends_on이면 DB 준비까지 보장된다.	실행 순서와 readiness는 다르다. healthcheck와 retry가 필요하다.
로그를 보면 다 알 수 있다.	여러 서비스 로그를 연결할 request id/correlation id가 없으면 원인 추적이 느려진다.

Evidence Note 양식

```
# Week 3 Evidence - Docker Compose로 전체 서비스 실행

## Service checked
- Service:
- Command:
- Expected:
- Actual:

## Dependency
- Upstream:
- Downstream:
- Config/env used:

## Failure or risk
- Symptom:
- Observed evidence:
```

- Fix or request:
- Recheck result:

평가 기준

기준	통과 조건
개념 이해	Docker Compose로 전체 서비스 실행을 서비스 경계/의존성/운영 증거 중 하나와 연결했다.
실행 증거	Compose 상태, 로그, HTTP 응답 중 하나 이상을 기록했다.
장애 분석	증상과 의심 서비스를 구분했다.
협업 기준	개발팀에 요청할 정보 또는 운영 문서에 남길 정보를 적었다.
다음 연결	Kubernetes에서 어떤 리소스로 이어질지 한 문장으로 설명했다.

공식/학술 근거 링크

- AWS Microservices on AWS, <https://docs.aws.amazon.com/whitepapers/latest/microservices-on-aws/microservices.html> - MSA를 여러 서비스와 API 기반 구조로 설명하는 공식 기준이다.
- Docker Compose Docs, <https://docs.docker.com/compose/> - 여러 service를 파일로 실행하고 관리하는 기준이다.
- Google SRE Book: Addressing Cascading Failures, <https://sre.google/sre-book/addressing-cascading-failures/> - 한 서비스 장애가 다른 서비스로 퍼질 수 있음을 설명하는 근거다.

다음 연결

다음 교시는 같은 실습 앱에서 다른 service boundary를 본다. 오늘 만든 evidence note는 서비스 목록, 의존성, health, logs를 누적하는 운영 문서가 된다.